

Reference architecture for open, maintainable and secure software for the operation of energy networks

Andre Goering¹ ✉, Juergen Meister¹, Sebastian Lehnhoff¹,
Peter Herdt², Martin Jung³, Matthias Rohr⁴

¹OFFIS, Oldenburg, Germany

²Main-Donau Netzgesellschaft, Nuremberg, Germany

³develop-group, Erlangen, Germany

⁴BTC AG, Oldenburg, Germany

✉ E-mail: andre.goering@offis.de

Abstract: Regulatory effects, business pressure, and the transformation to smart grids foster the need for up-to-date software systems to manage and operate the grid operators' electric power grids. The complexity of these systems has grown over decades. This makes enhancements and development of new functionalities in existing systems cost intensive, vendor/system specific and often prevents meeting time to market and quality requirements. Public interfaces and open data formats allow the development of enhancements and new functionality as reusable modules by third parties, thus enabling the integration of best-of-breed systems in the system landscape at grid operators. A reduction of system complexity is a precondition to develop such reusable modules while meeting time to market and quality requirements in critical infrastructure. This is accomplished by defining reference architecture with a common architecture framework, common processes, and quality standards.

1 Introduction

The steadily growing integration of decentral renewable energy resources, regulatory effects, business pressure in the unbundled energy sector, and the transformation to smart grids lead to many new requirements for software systems of grid operators to manage and operate their electric power grids. The complexity of existing systems has grown over decades: Each IT system (e.g. distribution management system [DMS]/supervisory control and data acquisition [SCADA], enterprise resource planning [ERP], customer relationship management [CRM], geographic information system [GIS]) only holds parts of relevant grid data. Via direct coupling between these systems, data are made accessible specifically for each business process and proprietary at each grid operator/vendor [1]. A net of point-to-point connections leads to dependencies between the systems that are unmanageable. Each of the named systems has a 5–15 years interval of major updates. The upgrade projects are highly complex and cost intensive because of a steadily growing range of (sometimes customer-specific) adaptations and extensions.

This results in a vendor lock-in of grid operators to their system vendors and requires enormous effort for integration, thus binding development capacities needed for new development or updates forced by regulation authorities.

These problems are addressed by a consortium called openKONSEQUENZ (<http://www.openkonsequenz.de>) (oK), whose goal it is to significantly reduce maintenance costs of their systems' landscape by decreasing system complexity and vendor dependency as well as increasing software quality, usability, and enhancing safety and security in critical infrastructures. According to the smart grid architecture model (SGAM) [2] the consortium mainly focuses on the development of new functionality in the higher hierarchical zones of operation and enterprise in the domain of power distribution as shown in Fig. 1.

The oK consortium brings together German and Dutch distribution system operators (DSOs) supplying over 15 million German households with electrical power and 5.7 million Dutch customers with gas and power, software vendors, service

providers, and researchers. It was started 2013 with the idea of developing open source software to solve the problems explained above. The consortium is organised in the Eclipse Foundation structure as driver members (a number of German DSOs), user members (DSOs with focus on development), service providers (including software vendors), and guest members (universities and research institutes, interested service providers, a Dutch DSO).

This paper discusses the reference architecture subsuming quality and architecture standards, the development process, and a project adoption process developed by oK. Architecture and quality standards have already been published in [4] but are also included in this paper for the sake of conceptual clarity. Section 2 describes the related work. Sections 3 and 4 discuss the reference architecture and its evaluation and evolution while Section 5 sums up the work and gives an outlook on future projects.

2 Related work

In the domain of the smart grid, the SGAM is the reference architecture for describing main actors and interactions with a presentation schema using the NIST Conceptual Model and the GridWise Architecture Council (GWAC) Stack as basis [2]. The GWAC Stack shows that interoperability has to be discussed on different levels (e.g. network, syntactic, semantic, procedures, or objectives). So, ensuring interoperability and making software development vendor independent and faster, while keeping software quality, leads to questions for standards on data exchange and architectures for a combination of modules of different vendors. These fields are discussed briefly in the context of the electricity domain in the following sections.

2.1 CIM standards – IEC 61970, 61968, and 62325

Interoperability issues in the field of oK are mainly addressed by the common information model (CIM) [3].

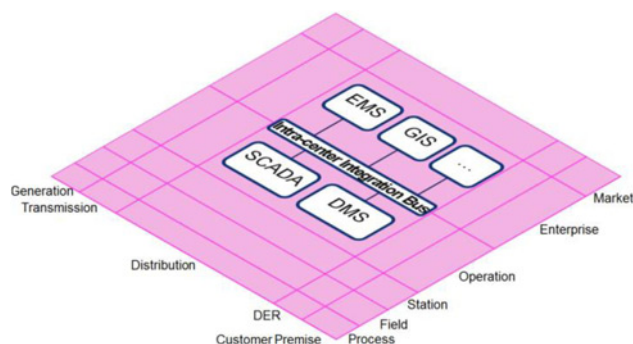


Fig. 1 SGAM classification of oK (see also [3])

The Electric Power Research Institute (EPRI) started developing the CIM in the 1990s to solve vendor lock-in at energy management systems (EMSs) [5]. Since then, it was developed further by the International Electrotechnical Commission (IEC) as a series of standards for information exchange: IEC 61970 for EMS, IEC 61968 for distribution management, and IEC 62325 for energy markets. Core of the CIM is a semantic data model for information exchange in and between electric utilities. The CIM data model describes all necessary structures/elements of electricity networks, their relationships and multiplicities, their semantical meaning, and their syntactical values from the point of view of the IEC. Although it is still under development to cover the whole (and future) extent of data in electrical information modeling, usage of CIM as a foundation for information interchange is superior to proprietary approaches. The European Network of Transmission System Operators for Electricity (ENTSO-E) uses the CIM as basis for their Common Grid Model Exchange Standard (CGMES) to exchange grid models between different transmission system operators (TSOs) and achieving interoperability between TSOs software systems. We expect that future model exchanges between TSOs and DSOs for e.g. power grid stability calculations underlie the same standard.

2.2 Reference architectures

Reference architectures are proven, generic software architectures for specific application domains and apply across product and organisational borders.

Enterprise application integration (EAI) [6] deals with business-critical applications. EAI defines patterns on how to integrate such applications as well as newly developed advanced components in order to create complex systems. In such systems, point-to-point connections are not appropriate to facilitate interoperability and maintainability. A central message broker such as an enterprise service bus (ESB) is a more suitable solution. It moves messages from any type of application to any other, changing the format of messages to meet target systems' requirements.

A service-oriented architecture (SOA) is derived from EAI. SOA has several layers: operational systems, enterprise components, services, business process choreography, and presentation [7]. SOAs main pattern is the combination and recombination of independent services, which base on existing enterprise components, to a business process choreography. The SOA layers match to the energy domain in the following way: existing IT-systems and sensor systems of network operators are operational systems. Enterprise components are built upon that first layer to enable e.g. DMS, SCADA, ERP, as well as CRM and create a second layer. In an SOA, on top of that layer, services provide single or combined functionality, which can be reorganised in business processes in the next layer and get presented in a top layer. The integration functionality and cross-cutting concerns like security, service management, and monitoring are orthogonal to these layers.

The open smart grid platform (<https://smartsocietyservices.com/osgp/>) (OSGP) is a platform for open, generic, scalable, and

independent 'Internet of things' (IoT) services. It provides information on widespread sensors like a SCADA kernel does to a DMS/SCADA and can be used as basis to get data from the field. According to the SOA concept, OSGP can be located in the enterprise layer.

2.3 Quality standards in agile development

Software quality [8, chapter 10] is essential in the development of long-living and safety/security-critical software systems e.g. for critical infrastructure. In particular, maintainability, confidentiality, integrity, and availability are important for oK software. Quality standards have been put in place to standardise the meaning of software quality with respect to said quality attributes (e.g. by BSI [9], BDEW [10]). These quality standards mainly focus on the operation of software and not on the development.

Agile methods have become a mainstream in software engineering [8, Section 4.4], and are also applied successfully in safety-critical environments such as critical infrastructure. As one of the specific methods, Scrum has proven to be capable of delivering fast results with high quality. The low project management overhead of Scrum and the self-organising team culture of all agile methods make Scrum the method of choice for open source development.

Intensive research has been done in comparison of closed source and open source. There is no verifiable general advantage of one over the other in terms of safety and security [11], but open source has other positive aspects, e.g. white box components, adaptability, extendibility, worldwide peer reviews, and easy reuse without legal issues.

3 Ok reference architecture

The oK consortium drives modularisation of DMS functionalities and adopts the benefits of the SOA approach. Applications and components are connected using an ESB and the interfaces of the services use a unified data model called oK-CIM-Profiles which base on the CIM standard. A major goal of oK is to provide an ecosystem in which development of independent modules and their integration in critical infrastructure are standardised. We assume, the enforcement of the reference architecture for software in the electricity domain is essential to enable interoperability of such modules. The reference architecture as well as basic open source platform services are developed to ease single-vendor-independent adoption of new modules in an ecosystem and allow long-term support.

3.1 Multilayer and technical architecture

Existing systems (possible) externally developed modules, oK user modules, and oK platform modules (core modules and domain modules) interact on the basis of standardised interfaces (the previously mentioned oK-CIM-Profiles) and run on an underlying system following the reference architecture concept.

Fig. 2 shows the oK multilayer architecture, which provides a general structure to ensure reusability, integratability, modularisation, and extendibility. Each module, component, system, and adapter has to be located at some point in this architecture (e.g. shared backend services in the platform layer and GIS, DMS, and ERP in the source system layer). Platform modules provide reusable basic functionality to multiple user modules and organise tasks such as source system data access. Platform modules are categorised into domain modules and core modules: Core modules provide services for cross cutting concerns in a standardised way, while domain modules provide specific services to the domain of operating power grid systems. User modules implement the use cases of end users. The modules communicate using the APIs shown in Fig. 2.

A technical architectural view is shown in Fig. 3. This shows that oK makes extensive use of open source technology to implement the modules. A typical oK application (i.e. a user module and required

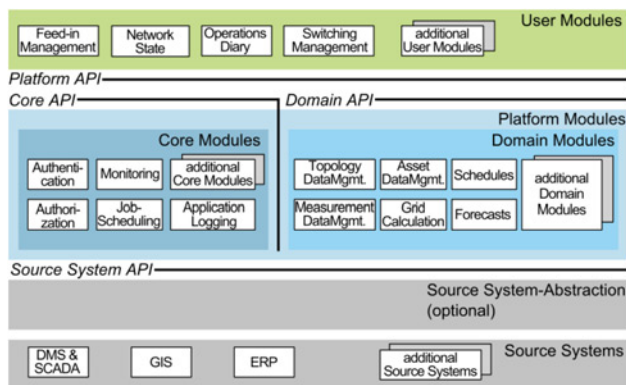


Fig. 2 oK multilayer architecture

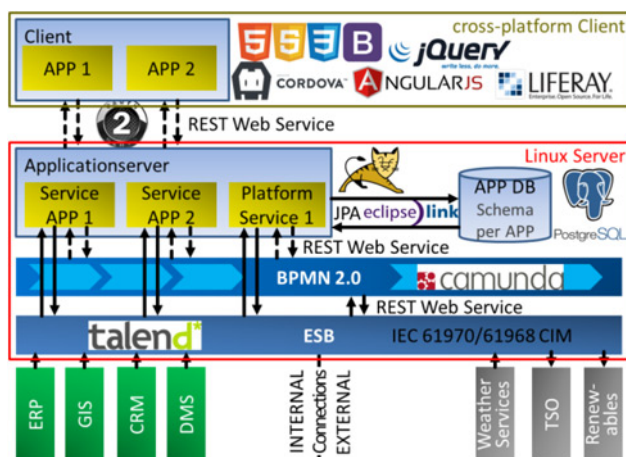


Fig. 3 Technical architecture

platform modules) is implemented in Java, has a Web interface, and stores own data in a PostgreSQL database. Following the microservice paradigm, a concrete implementation may use other technologies, such as for instance other database management systems.

3.2 Quality standards

The oK platform consists of open source software modules, developed by independent parties using an agile methodology. To engineer and safeguard quality requirements – foremost the security goals and maintainability – of all modules as well as the integrated platform, rigorous quality standards for the software development are mandatory.

oK defines its quality standards in three categories: code quality, design quality, and product quality.

Code quality is maintained by defining

- a set of coding guidelines,
- file naming conventions,
- configuration management conventions,
- build, package, and test mechanisms,
- runtime monitoring and diagnosis functions common for all modules.

The coding guidelines and the common conventions ensure conformity of the modules developed by independent organisations. Central elements of quality assurance on code level are static analysis, automated testing, and dynamic analysis. These mechanisms are implemented by reviews (see below) and by nightly builds on a continuous integration system which includes

static analysis (enforcing the coding guidelines), unit tests, and code coverage analysis.

Design quality is maintained by defining a set of design documents common for all modules. The central design document of each module is its architecture concept. The outline and contents of these documents are defined. A test specification document is also required for each module, defining integration test cases that are also run during the nightly builds. Design quality is maintained by a peer review setup. Design documentation – as well as the code itself – of each module is examined by a third party, typically the architects and developers of another module.

Product quality is maintained by using a reference installation environment ('QA environment'). After each sprint, the module is deployed into an oK platform installation in the QA environment. Each module has to produce a test specification and a validation concept to describe the test steps to be performed on the QA environment. As far as possible, these test steps should be automated, but manual tests will be required on product level. The manual tests are executed by the product owners, at least once at sprint end, before they accept a feature/user story. The product documents are also subject to peer review. The intensity of reviews to ensure quality is determined by the classification of a module in terms of criticality and complexity. The documents will be created and filled according to the agile development method. Availability of the document contents relevant for a feature/user story is part of the 'done' criteria. This helps keeping the documents up to date, and also helps to keep the review scope in each sprint small.

The quality rules are independent of technology as far as possible and may be adjusted to technologies applied in oK now or in the future.

3.3 User interface style guide

In critical situations, users need to identify and select their options fast. To get an adequate reaction time, users need to be comfortable with the user interface (UI) even though different modules may have different ones. To address this and further end-user requirements, the EN ISO 9241-110 was chosen as a foundation of oK's UI style guide. The style guide is mandatory, thus ensuring the same user experience and comfortability for different modules and also reducing the need for training on each UI.

3.4 Development process

To fulfil oK's vision and create a reference platform for high-quality grid operator software applications, a vibrant ecosystem of developers and users is necessary, and oK has to exercise some sort of control over the open development. Control is necessary in two ways, to steer the development of the platform features and to control the quality of the platform.

oK issues requests for tenders in order to have vendors develop features (user and platform modules as shown in Fig. 2) of the platform. By prioritising such tenders, the development of the platform can be guided. Also, oK requires vendors to state the dependencies to the reference platform modules. This enables oK to enforce dependency management early on, and ensure availability of the platform modules.

The oK community relies on three groups of participants:

- *Users (and their integrators)* who deploy the oK platform, integrate it in their IT systems, and operate it. Typically, they contribute to the community by reporting issues with the platform and requesting additional features. An example for users is the grid operators. The tenders mentioned above are driven by the users.
- *Contributors* who fix issues or work on features. Typically, they contribute substantial amounts of code, over a limited time. An example for contributors is researchers or software service providers.
- *Committers* who exercise QA on the work of the contributor. Additionally, they perform architecture work and maintenance.

Typically, they contribute by prioritising the work on features and issues, over extended periods of time. Examples for committers are the oK driver members, operators, and vendors.

Finally, to centralise all work on the oK platform, a host for the community is required. There are several such hosts e.g. the apache foundation, GitHub, or the eclipse foundation. They vary in the amount of services they provide, as well as the different open source licenses they support. oK's primary criteria for the selection of a host environment was the availability of an intellectual property (IP) check service and the support of a license that allows the commercial usage of the platform.

oK chose the eclipse foundation as a host. They offer an elaborate IP management process, and its license; the Eclipse public license (EPL) allows a variety of business models. Also, it offers a development environment including source code repositories and continuous integration services, and an easy integration to many other open source projects e.g. IoT.

3.5 Project adoption process

The development as open source enables everybody to contribute new functionality. In existing modules, according to Section 3.4, the committers are responsible for checking contributions. For contribution of whole modules in the landscape of oK, there are different levels of conformity:

- (a) *oK-ready* – a vendor labels his software to be compatible to oK interfaces without further checks,
- (b) *oK-ready** – the vendors software is tested for API compatibility by oK,
- (c) *oK-UI-styleguide-conformity*,
- (d) *oK-architecture-conformity*,
- (e) *oK-quality-conformity*,
- (f) *full-oK* – all above mentioned are, approved by oK, published as open source in an oK eclipse project.

Closed source commercial modules of a vendor may also be classified as levels b to e by certification authorities that are approved by oK. For level f, the source code needs to be open. When adopting software as *full-oK*, a new form of lock-in could arise, and has to be avoided. This may be by pre-dominance of technical solutions in the case of large components donated by industry organisations, and also by unique know-how in contributions from the scientific community. The quality standards mentioned above serve to mitigate this risk. Also, there are cycles of checks (by oK) and modifications (by module contributors or oK) to discover and correct violations of the reference architecture.

4 Evaluation

The evolution of the reference architecture is part of the overall development process, described in Section 4. So oK drives the reference architecture development parallel to module development. After an initial draft architecture, created in a pilot project for feed-in-management (a German demonstrator can be found on <http://demo.openkonsequenz.de>), the architectural development is performed in oK's architectural and quality committees (AC/QC) with respect to future modules. It will be adapted and supplemented to future demands of module development. For example, in the current awarding process of the module *Operation Diary*, details on authentication and

authorisation will be examined by the module developers and discussed by the AC/QC not only for the operation diary but for all future modules. The novel architecture concepts are evaluated in sprint reviews, the project reviews and regular discussion, as well as in a research project on integration of big data technologies in the operation of energy networks called *NetzDatenStrom*, and the integration projects, where each newly developed oK module is adopted to individual DSOs system landscapes. Initial evaluation in the pilot project shows a barrier in using CIM by DSOs, mainly due to unfamiliarity with this set of standards. As discussed above, application of the CIM standards is essential.

5 Summary

The oK consortium drives reference architecture development in the field of electricity network management and operation to overcome the existing vendor lock-in, monolithic systems, and system complexity that hinders development of new, needed functionalities for smart grids. Therefore, the consortium uses CIM standards to ensure interoperability and applies open source development methodology to support both an open multi-vendor ecosystem and quality assurance.

The oK specifies multilayer and technical architecture, the quality assurance, as well as guidelines for UIs and processes for the development and adoption of projects. Developed under the EPL, the modules are kept for being modifiable and usable in DSOs system landscapes. Results of further development of oK modules by vendors are shared with the community. A foundation of platform modules is build up in single module projects, while quality assurance is in place over all projects. In the long term, modules implementing new functionalities for the users can rely on a growing number of platform modules, maintained as open source software. Novel functionality can be developed as open source by the consortium as well as closed source by commercial vendors with the possibility of easy integration in the landscape of electricity grid operators reaching more software efficiency and reliability.

6 References

- 1 ARGE KONSEQUENZ: 'Machbarkeitsstudie – Konsortiale Softwareentwicklung auf der Basis von Open-Source-Software, ARGE KONSEQUENZ', 2013
- 2 CEN-CENELEC-ETSI Smart Grid Coordination Group, 2012, Smart Grid Reference Architecture, CEN CENELEC
- 3 IEC: 'Smart grid standards map', IEC. <http://smartgridstandardsmap.com/>, 2014, accessed on 13 January 2017
- 4 Goering, A., Meister, J., Lehnhoff, S., *et al.*: 'Architecture and quality standards for the joint development of modular open source software for power grid distribution management systems'. Proc. 5th 'D-A-CH + Energy Informatics Conf.', OVE, Klagenfurt, Austria, 2016, vol. 84, pp. 36–39
- 5 Uslar, M., Specht, M., Rohjans, S., *et al.*: 'The common information model CIM – IEC 61968/61970 and 62325 – a practical introduction to the CIM' (Springer, Berlin, Germany, 2012)
- 6 Linthicum, D.S.: 'Enterprise application integration' (Addison-Wesley, Boston, USA, 2000)
- 7 Arsanjani, A.: 'Service-oriented modeling and architecture – how to identify, specify, and realize services for your SOA', IBM, Armonk, NY, USA, 2004
- 8 Bourque, P., Fairley, R.E.: 'Guide to the software engineering body of knowledge SWEBOK, version 3.0', *IEEE*, 2014, 3, pp. 1–335
- 9 Lefin, N.: 'Weiterentwicklung des IT-Grundschatzes – IT-Grundschatz-Profil für Open-Source-Software (GSProOss)'. Master thesis, Hochschule Landshut, 2010
- 10 BDEW: 'Requirements for secure control and telecommunication systems', Version 1.1, BDEW, Berlin, Germany, 2015
- 11 Schryen, G.: 'Is open source security a Myth?', *Commun. ACM*, 2011, **54**, (5), pp. 130–139